



University
of Glasgow

Lecture 3: Access Control: Authentication & Authorization

A WORLD
TOP 100
UNIVERSITY

Dr. Yutian Tang

Yutian.Tang@glasgow.ac.uk

WORLD
CHANGING
GLASGOW

THE SUNDAY TIMES
GOOD
UNIVERSITY
GUIDE
2024
SCOTTISH
UNIVERSITY
OF THE YEAR



University
of Glasgow

**Scan the QR code to
take your attendance**

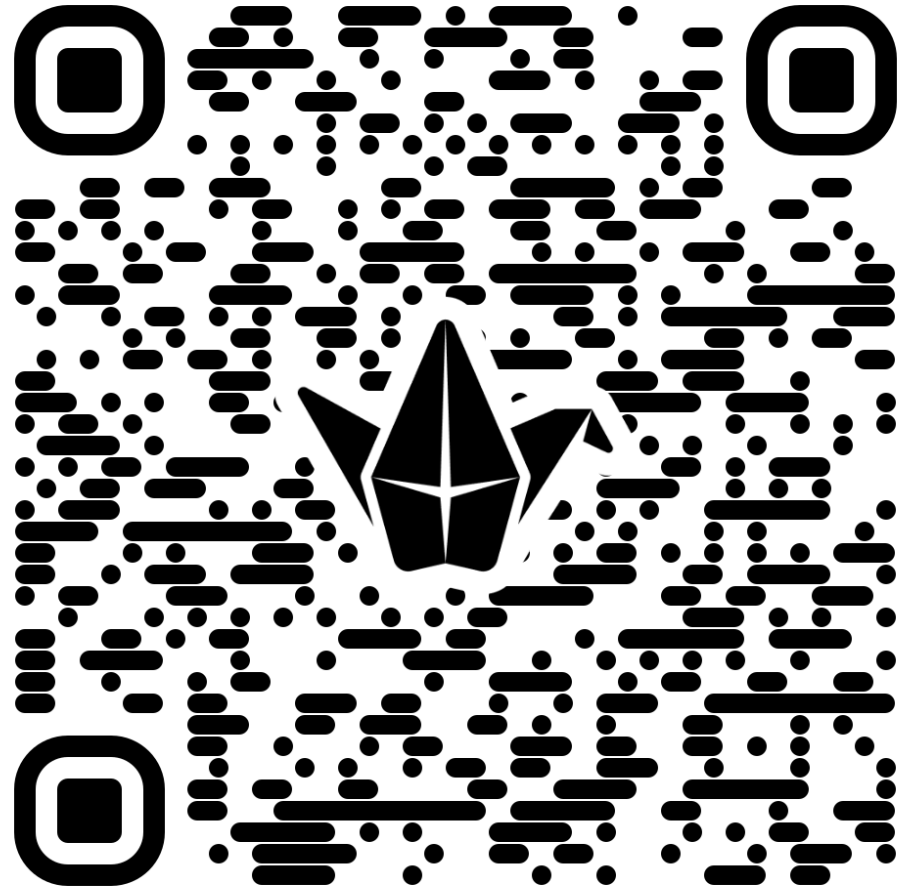
Note that this QR code is
only valid today (3:00 – 6:00
PM)





University
of Glasgow

:Padlet for Questions



- <https://padlet.com/yutiantang/sse-lecture-3-question-wall-p1j3r97ab7jq4sd6>

Please post any questions you have during the lecture on this Padlet. I will address them at the end.



University
of Glasgow

Outline

- Access Control
- Implement Digital Identity (Authentication)
- Authentication
- Applications of Access Controls



University
of Glasgow

Access Control

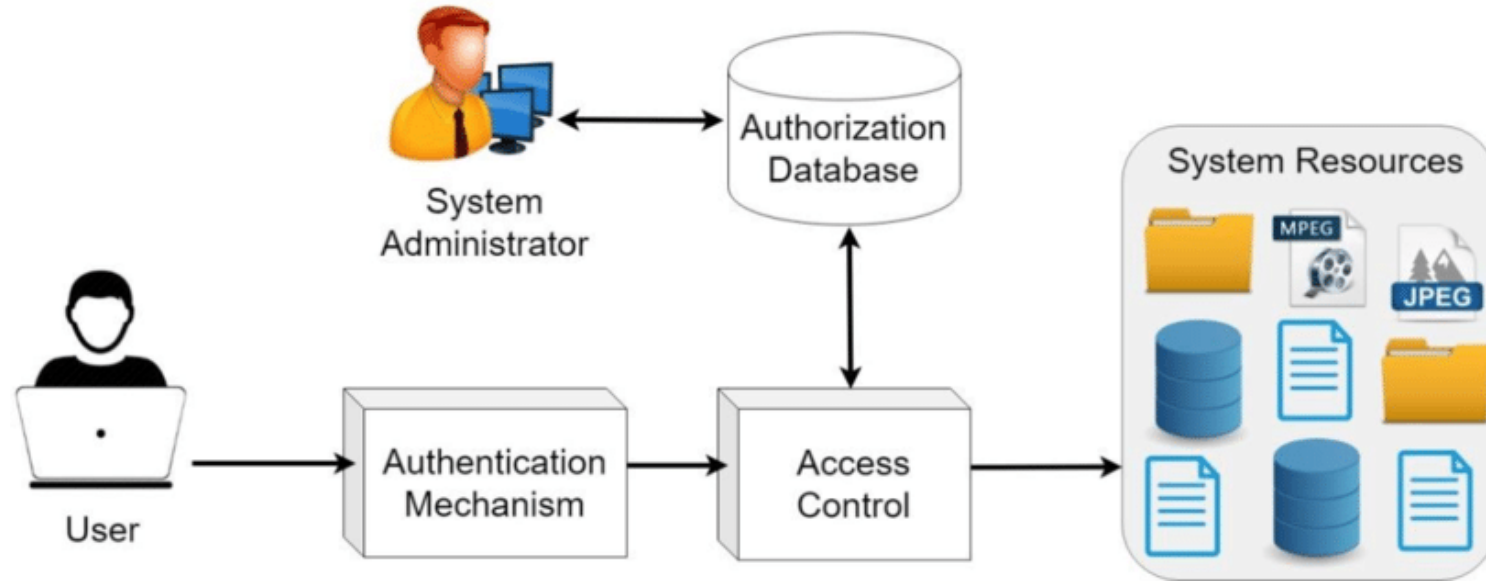


Access Control

- Access control involves identifying a user based on their credentials and then **authorizing** the appropriate level of access once they are **authenticated**.
- Passwords, pins, security tokens and biometric scans are all credentials commonly used to identify and authenticate a user.
- Once a user's identity has been **authenticated**, **access control policies** grant **specific permissions** and enable the user to proceed as they intended.

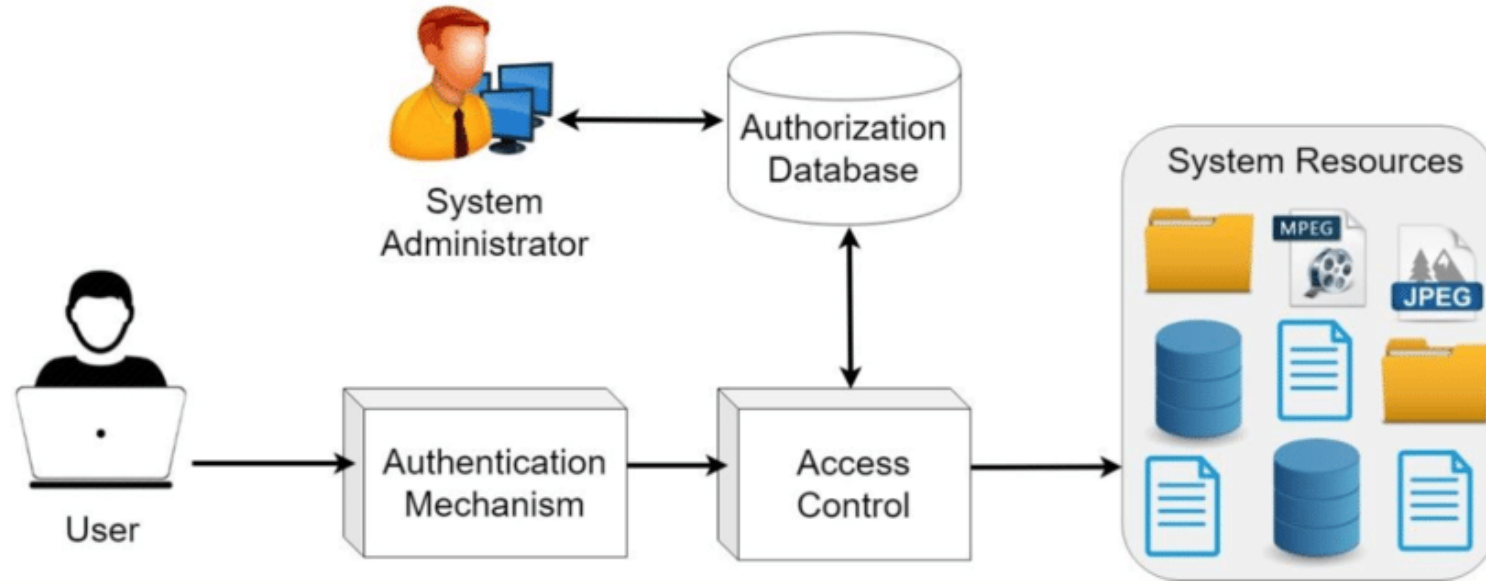


Access Control'2 (A Workflow)



1. **User:** The individual who requests access to system resources.
2. **Authentication Mechanism:** Verifies the identity of the user through a login process with a username and password.
3. **System Administrator:** Responsible for managing access privileges (e.g, inputting and updating the authorization database)

Access Control'3

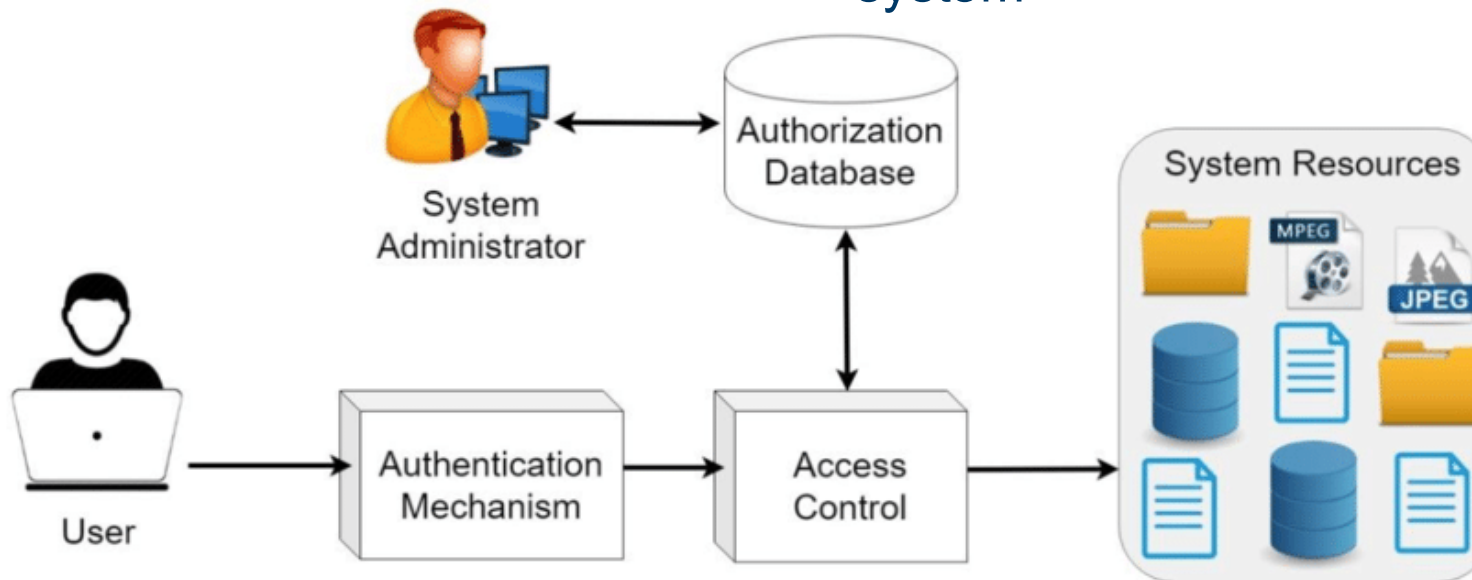


4. **Authorization Database:** Stores information about user privileges and access rights, which are managed by the system administrator.
5. **Access Control:** The system that uses the **authorization database** to grant or deny user requests based on their authenticated identity and access rights.
6. **System Resources:** The files, databases, applications, and other resources to which access is controlled. This is the final element that the user is trying to access.



Access Control'4

Authorization: determining whether an authenticated user or system **has the right to access specific resources or perform certain operations within a system**



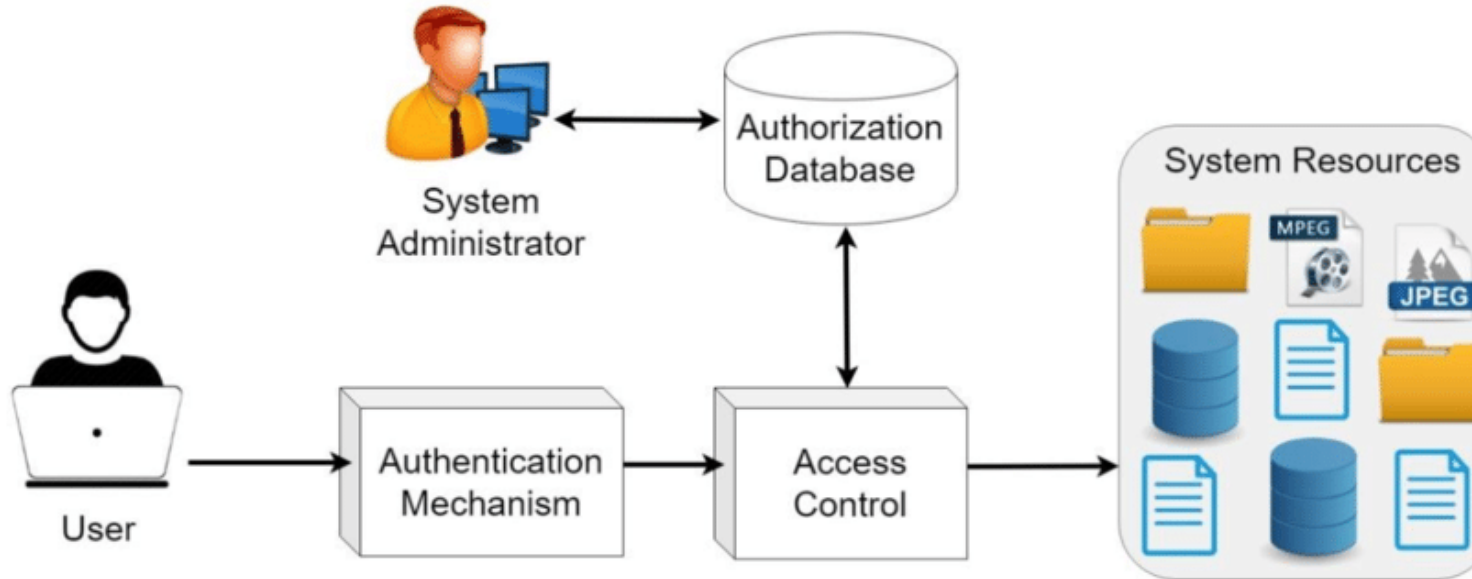
Authentication: Process of **verifying the identity** of a user or device.



Access Control'5

What you can do in the system?

Authorization



Authentication

Who you are/Whether you are allowed to used the system?



University
of Glasgow

Implement Digital Identity & Authentication



Authentication

- **Def:** Authentication is the process of **verifying that an individual or entity is who they claim to be.**
- It's a critical component in access control.
- It ensures only verified users or entities can access the software system.
- It's the first checkpoint in the security framework



Introduction to Digital Identity:

- **Def: Digital Identity** is a set of attributes and characteristics that uniquely identify a person or entity in the digital world.
- **Key Components**
 - **Username**
 - **Biometrics**
 - **Digital Certificates**
 - **Personal Identification Numbers (PINs)**
 - **Email Addresses**
 - ...



Lifecycle of Digital Identity

1. Enrolment/Registration:

The initial stage where an individual's identity is first captured and established within a system. This includes gathering Personally Identifiable Information (PII), setting up credentials, and registering biometric data.

- **Example:**

- A user signs up for an online banking service, providing his/her name, address, government-issued ID, and setting up security questions.



Lifecycle of Digital Identity'2

2. Maintenance/Updates:

The ongoing process of ensuring the digital identity remains accurate and secure.

This involves updating personal details, changing passwords, and refreshing biometric data as needed.

- **Example:**

- A user updates his/her password after a system prompt for periodic change or revises his/her contact information after moving to a new house.



Lifecycle of Digital Identity'3

3. Deactivation/Deletion:

The final phase where a digital identity is either temporarily disabled or permanently removed from a system.

Deactivation may occur due to inactivity or by user choice, while deletion typically follows a request to erase personal data in accordance with privacy regulations.

- **Example:**

- A user deactivates their social media account for a digital detox or requests deletion of their online shopping account following privacy concerns.



Authentication Methods

- **Representative Approaches:**

1. Password-Based Authentication
2. Token-Based Systems (OneTimePassword/OTP)
3. Biometric Verification
4. CAPTCHA and reCAPTCHA



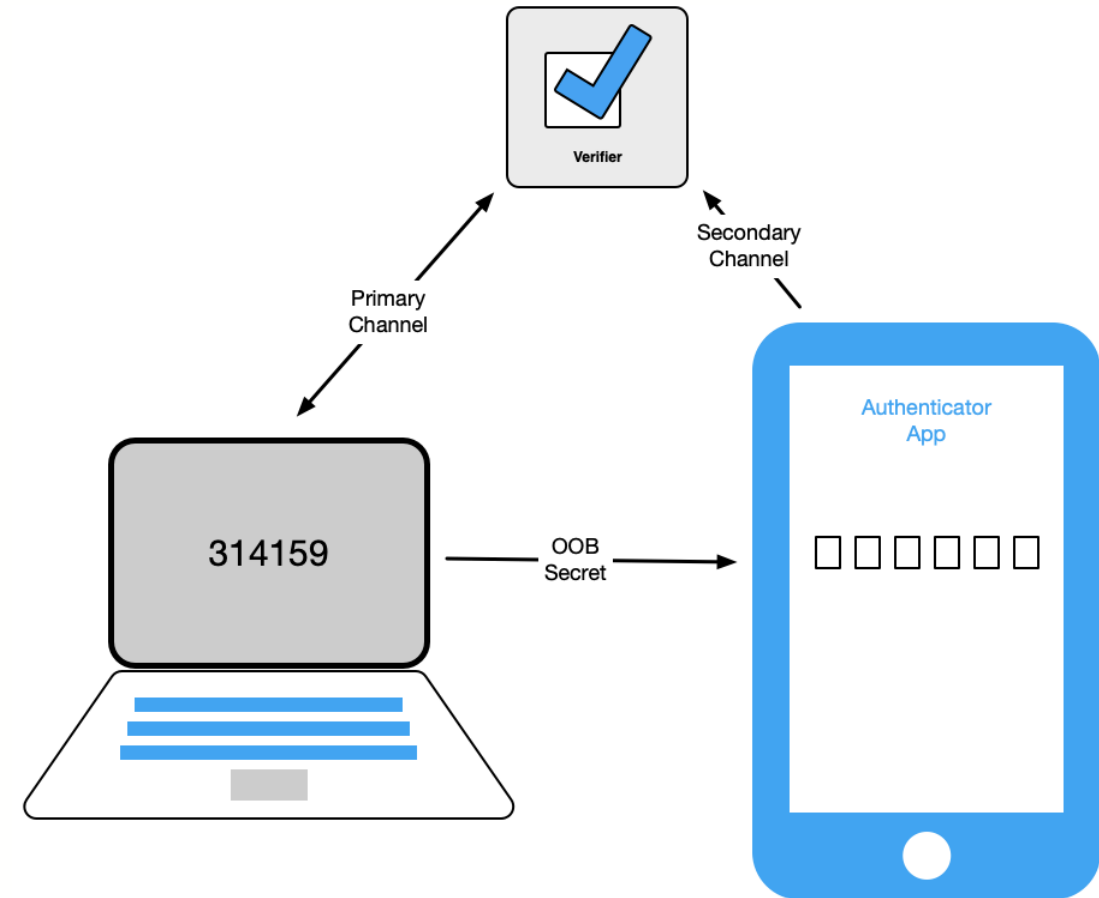
Authentication Levels

- **AAL 1 (Authenticator Assurance Level 1)**
 - **Single-Factor Authentication (SFA)**
 - The simplest form of authentication that requires only one factor
 - E.g. Password, PIN



Authentication Levels

- **AAL 2**
 - **Two-Factor Authentication (2FA) /Multi-Factor Authentication (MFA)**
 - Requires two distinct factors for authentication.
 - E.g. Password + Security Token/App/...
 - Use Case Example: Banking applications, email accounts, and online shopping.





Authentication Levels

- **AAL 3**
 - **Multi-Factor Authentication with Advanced Requirements (AAL3)**
 - Requires at least two out of three authentication factors, including the use of a hardware-based authenticator.
 - Suite for high-security environments and sensitive data access.



Password-Based Authentication

- **Definition and Significance:**

- A method that verifies identity based on 'something the user knows'—typically a password.
- Essential for protecting personal data against unauthorized access

- **Design Considerations:**

- Appropriate hashing algorithms for storing passwords.
- Defending against common attacks like brute-force.
- Considering technological advances and alternatives to passwords.



Password-Based Authentication: Challenges and Best Practices

- **Challenges:**

- Passwords must remain a shared secret, but they're prone to being guessed or stolen.
- Risk of 'credential stuffing' from reused passwords across services.



Password-Based Authentication: Challenges and Best Practices

- **Best Practices:**

- Use strong, unique passwords and avoid common passwords.
- Implement secure password reset processes.
- Rate limit login attempts to prevent attacks
- Use CAPTCHA to prevent automated bot registrations.
- Allow users to display their password while typing to prevent entry errors.

Please check the box below to proceed.

☐

I'm not a robot



reCAPTCHA
Privacy - Terms

- **Security Measures:**

- Hashing passwords with algorithms, and not storing them in plaintext.
- Protect login pages with HTTPS to prevent interception of passwords



Common Passwords

- <https://cry.github.io/nbp/>

What is NBP?

NIST Bad Passwords, or NBP, aims to help make the reuse of common passwords a thing of the past. With the release of **Special Publication 800-63-3: Digital Authentication Guidelines**, it is now recommended to blacklist common passwords from being used in account registrations.

NBP is intended for quick *client-side* validation of common passwords only. It is still advisable to check server side if the password is not common.

From [Naked Security @ Sophos](#):

Check new passwords against a dictionary of known-bad choices. You don't want to let people use ChangeMe, thisisapassword, yankees, and so on.

Demo

123345

Your password is **common**.

This demo uses SecList's 1,000,000 most common password list.

The following link contains the most commonly found passwords

- <https://github.com/danielmiessler/SecLists/tree/master/Passwords/Common-Credentials>



Token-Based Systems

- **Definition:** A token is an item or data that physically or digitally authenticates identity. It can be a hardware device or a digital certificate.
 - **Physical Tokens:** Like a chip and PIN card, which provides evidence of the user's presence.
 - **Digital Tokens:** Such as OTPs or digital certificates used in electronic wallet apps. (OTP: One-Time Password)



Digital Tokens (OTP)

- **OTP:** One-time Password
- OTPs are unique codes that are valid for one session or transaction only.
- They are generated algorithmically and sent to a user's mobile device via SMS, email, or an authentication app.
- Upon login attempt, the system prompts for the OTP, ensuring that only the person with the device can complete the authentication process.



Designing an OTP System:

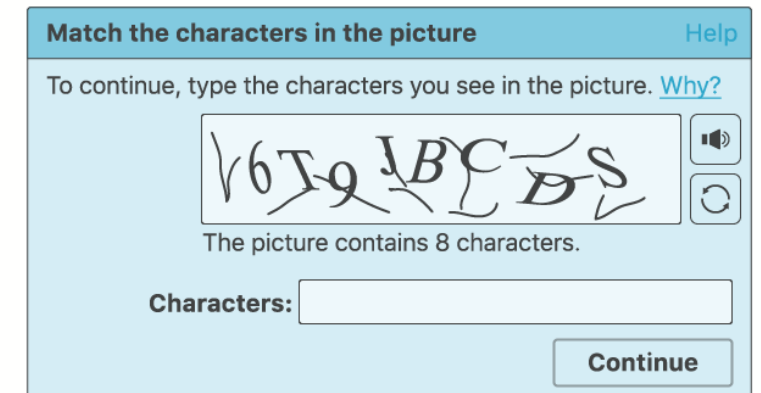
- **Generation:** Use a secure random number generator; often time-based (TOTP) or event-based (HOTP).
- **Delivery:** Ensure a secure channel for sending the OTP to the user, like encrypted SMS or push notifications.
- **Validation:** The server validates the OTP against the expected value and only allows access if there is a match within the appropriate time or attempt window.
- **Security:** Implement rate-limiting to prevent brute force attacks and consider fallbacks for delivery failures.



CAPTCHA

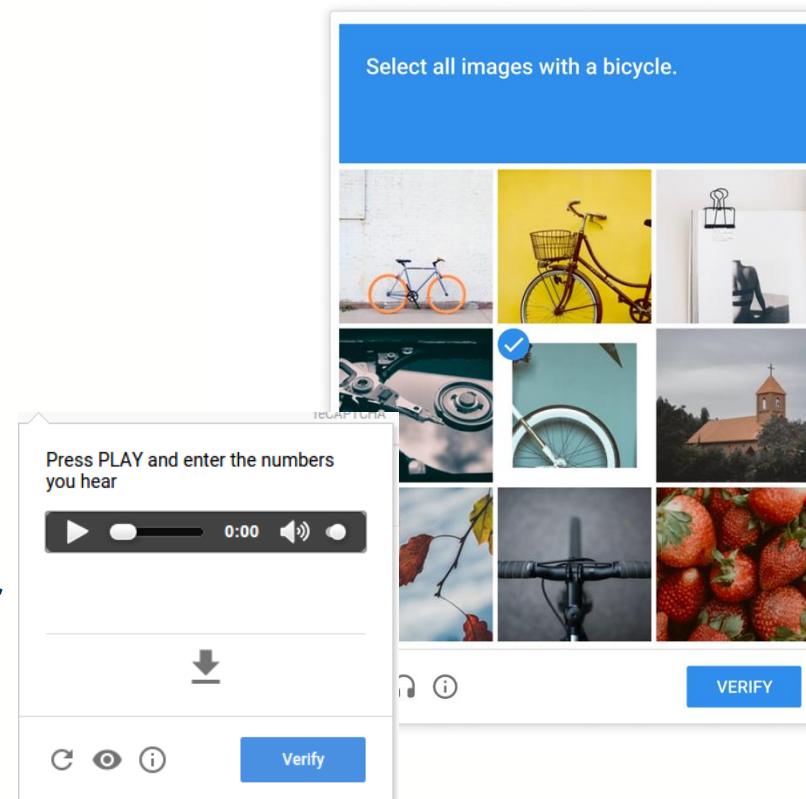
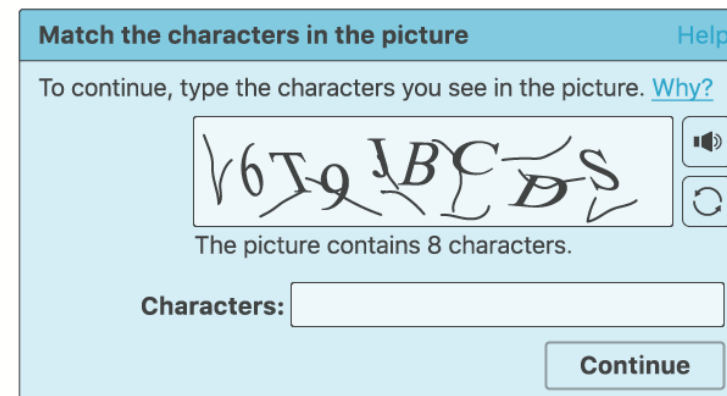
- **CAPTCHA**

- Completely Automated Public Turing test to tell Computers and Humans Apart (CAPTCHA)
- CAPTCHAs are tools you can use to differentiate between real users and automated users
- Was developed in early 2000s.



Types of CAPTCHA

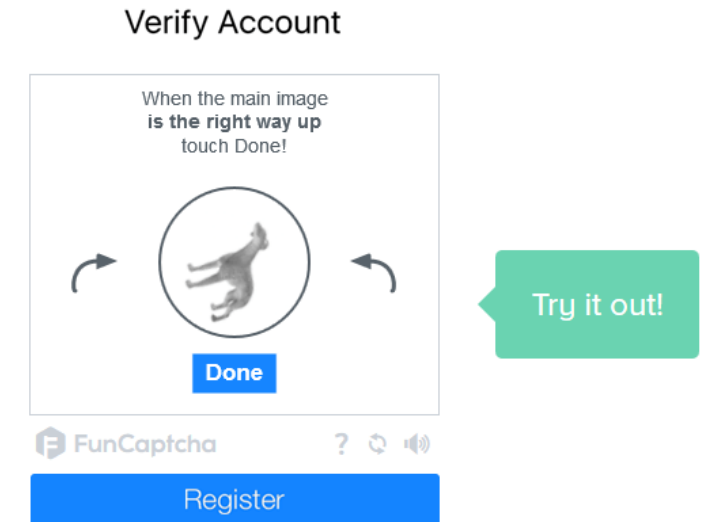
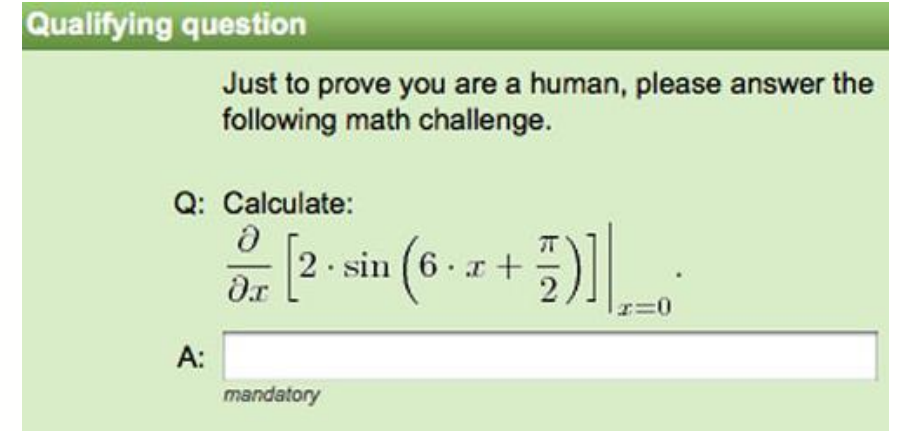
- **Text-Based** CAPTCHA: These CAPTCHAs can use known words or phrases, or random combinations of digits and letters
- **Image-Based** CAPTCHA: These CAPTCHAs use recognizable graphical elements, such as photos of animals, shapes, or scenes (e.g., “Select all images with traffic lights”).
- **Audio** CAPTCHA: Audio clips of a series of letters or numbers which a user then enters.





Types of CAPTCHA

- **Math** CAPTCHA: Simple math problems that users must solve.
- **Interactive** CAPTCHA: Puzzles or games that require human interaction to solve.



Img: <https://brightdata.com/blog/brightdata-in-practice/how-to-bypass-captcha-using-web-unlocker>

Img: <https://www.ionos.co.uk/digitalguide/online-marketing/online-sales/captcha-codes-and-images-for-spam-protection/>



University
of Glasgow

A WORLD
TOP 100
UNIVERSITY

Secured Software Engineering (M) & Secured Software Engineering for MSc (M)

Lab: Access Control: One-Time Password

WORLD
CHANGING
GLASGOW

THE SUNDAY TIMES
GOOD
UNIVERSITY
GUIDE
2024
SCOTTISH
UNIVERSITY
OF THE YEAR



One-Time Password

- Programming Language: Python
- Library:
 - PyOTP - The Python One-Time Password Library
 - <https://pyauth.github.io/pyotp/>
- Download the Authentication App:
 - Google Authenticator
 - <https://play.google.com/store/apps/details?id=com.google.android.apps.authenticator2> (for Android Users)
 - <https://apps.apple.com/us/app/google-authenticator/id388497605> (for iOS Users)
 - Search 'Google Authenticator' for other app stores



University
of Glasgow

Setup

The screenshot shows the PyCharm IDE interface. The 'Settings' dialog is open, and the 'Python Interpreter' section is selected. A red arrow points to the '+' button in the 'Package' list. The 'Available Packages' dialog is also open, showing a search for 'pyotp' and the package 'pyotp' selected. The 'Install Package' button is visible at the bottom.

1. Settings
2. Click '+'
3. Search 'pyotp'
4. Install the package



University
of Glasgow

Time-based OTP (One-time Passwords)

Time-based OTP (One-time Passwords)

```
import pyotp
import time

if __name__ == '__main__':
    totp = pyotp.TOTP('base32secret3232')
    current_otp = totp.now()
    print(current_otp)
    ver_result = totp.verify(current_otp) # => True
    print(ver_result)
    time.sleep(30)
    ver_result = totp.verify(current_otp) # => False
    print(ver_result)
```


Time-based OTP (One-time Passwords)

```
import pyotp  
import time
```

```
if __name__ == '__main__':  
    totp = pyotp.TOTP('base32secret3232')
```

This line creates a TOTP object.

It uses 'base32secret3232' as the secret key.

This secret key is usually provided by a service provider (like a website) and associated with a user's account.

the pyotp library generates a one-time password based on the current time and a pre-set secret key (in this case, 'base32secret3232').

Time-based OTP (One-time Passwords)

```
import pyotp
import time
```

```
if __name__ == '__main__':
    totp = pyotp.TOTP('base32secret3232')
    current_otp = totp.now()
```

totp.now() generates the OTP for the current time point.

This value typically changes every 30 seconds, as the TOTP algorithm is based on 30-second time steps.

<https://github.com/pyauth/pyotp/blob/develop/src/pyotp/totp.py>

```
class TOTP(OTP):
    """
    Handler for time-based OTP counters.
    """

    def __init__(
        self,
        s: str,
        digits: int = 6,
        digest: Any = None,
        name: Optional[str] = None,
        issuer: Optional[str] = None,
        interval: int = 30,
    ) -> None:
        """
        :param s: secret in base32 format
        :param digits: number of integers in the OTP. Some apps expect this to be 6 digits, others support more.
        :param digest: digest function to use in the HMAC (expected to be SHA1)
        :param name: account name
        :param issuer: issuer
        :param interval: the time interval in seconds for OTP. This defaults to 30.
        """
        if digest is None:
            digest = hashlib.sha1

        self.interval = interval
        super().__init__(s=s, digits=digits, digest=digest, name=name, issuer=issuer)
```


Time-based OTP (One-time Passwords)

```
import pyotp
import time

if __name__ == '__main__':
    totp = pyotp.TOTP('base32secret3232')
    current_otp = totp.now()
    print(current_otp)
    ver_result = totp.verify(current_otp) # => True
    print(ver_result)
```

This line verifies whether the provided OTP (in this example) is correct. Since this OTP is generated within the current time step, the verification result is True.

Time-based OTP (One-time Passwords)

```
import pyotp
import time

if __name__ == '__main__':
    totp = pyotp.TOTP('base32secret3232')
    current_otp = totp.now()
    print(current_otp)
    ver_result = totp.verify(current_otp) # => True
    print(ver_result)
    time.sleep(30)
    ver_result = totp.verify(current_otp) # => False
    print(ver_result)
```

`time.sleep(30)` makes the program wait for 30 seconds. After these 30 seconds, the original OTP has expired because it has moved into the next 30-second time step.

Thus, using the same OTP to call `totp.verify` again will return `False`, indicating a failed verification.



University
of Glasgow

Example

1. Phone number
2. Generate OTP
3. Send verification code (OTP)
4. Verify

Login 

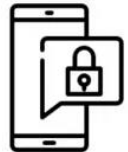
 Hey there!
Welcome

Enter phone number

 +910000000000



Login 

 Hey there!
Welcome

Enter verification code

____ _

00:28

[Change number](#) OTP sent to +919711949812 

Login 

 Hey there!
Welcome

Enter verification code

00:28

[Change number](#) OTP sent to +919711949812 



University
of Glasgow

Lecture 3B: Access Control: Authentication & Authorization

A WORLD
TOP 100
UNIVERSITY

Dr. Yutian Tang

Yutian.Tang@glasgow.ac.uk

We are resuming at 17:XX

WORLD
CHANGING
GLASGOW

THE SUNDAY TIMES
GOOD
UNIVERSITY
GUIDE
2024
SCOTTISH
UNIVERSITY
OF THE YEAR



University
of Glasgow

Counter-based OTPs (One-time Passwords)

Counter-based OTP (One-time Passwords)

```
import pyotp
```

```
if __name__ == '__main__':  
    hotp = pyotp.HOTP('base32secret3232')  
    counter_v0 = hotp.at(0) # => '260182'  
    counter_v1 = hotp.at(1) # => '055283'  
    counter_v1401 = hotp.at(1401) # => '316439'  
  
    # OTP verified with a counter  
    print(hotp.verify(counter_v1401, 1401)) # => True  
    print(hotp.verify(counter_v1401, 1402)) # => False
```


Counter-based OTP (One-time Passwords)

```
import pyotp
```

```
if __name__ == '__main__':  
    hotp = pyotp.HOTP('base32secret3232')  
    counter_v0 = hotp.at(0) # => '260182'  
    counter_v1 = hotp.at(1) # => '055283'  
    counter_v1401 = hotp.at(1401) # => '316439'
```

These lines generate OTPs for different counter values.

The `hotp.at(counter)` method generates an OTP for the given counter value.

For example, `hotp.at(0)` generates an OTP for the counter value 0, and `hotp.at(1401)` for the counter value 1401.

Counter-based OTP (One-time Passwords)

```
import pyotp
```

```
if __name__ == '__main__':  
    hotp = pyotp.HOTP('base32secret3232')  
    counter_v0 = hotp.at(0) # => '260182'  
    counter_v1 = hotp.at(1) # => '055283'  
    counter_v1401 = hotp.at(1401) # => '316439'  
  
    # OTP verified with a counter  
    print(hotp.verify(counter_v1401, 1401)) # => True  
    print(hotp.verify(counter_v1401, 1402)) # => False
```

The `hotp.verify(otp, counter)` method is used to validate an OTP. The first parameter is the OTP to be verified, and the second parameter is the counter value.

- `hotp.verify('316439', 1401)` returns `True` because '316439' is the correct OTP for the counter value 1401.

- `hotp.verify('316439', 1402)` returns `False` because '316439' is not the correct OTP for the counter value 1402.

Generating a Secret Key

A helper function is provided to generate a 32-character base32 secret, compatible with Google Authenticator and other OTP apps:

`pyotp.random_base32()`

Some applications want the secret key to be formatted as a hex-encoded string:

`pyotp.random_hex()` *# returns a 40-character hex-encoded secret*



University
of Glasgow

Google Authenticator Compatible

- PyOTP works with the Google Authenticator iPhone and Android app, as well as other OTP apps like Authy.
- PyOTP includes the ability to generate provisioning URIs for use with the QR Code scanner built into these MFA client apps.

Generate Secret Key and QR Code



```
def generate_qr_code():  
    # Generate a random secret key  
    #secret = pyotp.random_base32()  
    secret = "M6MX4TV2S4PMQYASP2DYUPS2UAG7O4UW"  
  
    # User's identifying information  
    user_identifier = "user@sse.com"  
    service_provider = "SSE"  
  
    # Generate a URI for Google Authenticator  
    uri = pyotp.totp.TOTP(secret).provisioning_uri(user_identifier, issuer_name=service_provider)  
    # Generate and display the QR code  
    qr_img = qrcode.make(uri)  
    qr_img.show()  
  
    return uri  
  
if __name__ == "__main__":  
    secret_key = generate_qr_code()  
    print(f"Your secret key is: {secret_key}")
```


Generate Secret Key and QR Code

```
def generate_qr_code():  
    # Generate a random secret key  
    #secret = pyotp.random_base32()  
    secret = "M6MX4TV2S4PMQYASP2DYUPS2UAG7O4UW"  
  
    # User's identifying information  
    user_identifier = "user@sse.com"  
    service_provider = "SSE"  
  
    # Generate a URI for Google Authenticator  
    uri = pyotp.totp.TOTP(secret).provisioning_uri(user_identifier, issuer_name=service_provider)
```

```
uri = pyotp.totp.TOTP(secret).provisioning_uri(user_identifier,  
issuer_name=service_provider):
```

Generates a URI that includes the secret key and the service provider's information. This URI is used by the Google Authenticator app to configure the TOTP for the user.



Generate Secret Key and QR Code



```
def generate_qr_code():  
    # Generate a random secret key  
    #secret = pyotp.random_base32()  
    secret = "M6MX4TV2S4PMQYASP2DYUPS2UAG7O4UW"  
  
    # User's identifying information  
    user_identifier = "user@sse.com"  
    service_provider = "SSE"  
  
    # Generate a URI for Google Authenticator  
    uri = pyotp.totp.TOTP(secret).provisioning_uri(user_identifier, issuer_name=service_provider)  
    # Generate and display the QR code  
    qr_img = qrcode.make(uri)  
    qr_img.show()  
  
    return uri  
  
if __name__ == "__main__":  
    secret_key = generate_qr_code()  
    print(f"Your secret key is: {secret_key}")
```

- `qr_img = qrcode.make(uri)`: Generates a QR code from the URI.
- `qr_img.show()`: Displays the QR code in a new window. This QR code can then be scanned by the Google Authenticator app.



University
of Glasgow

Generate Secret Key and QR Code



Google Authenticator and
other OTP apps



Verify the TOTP

```
import pyotp

def verify_totp(secret):
    # Create a TOTP object
    totp = pyotp.TOTP(secret)

    # Get the TOTP from the user
    user_totp = input("Enter your TOTP: ")

    # Verify the TOTP
    if totp.verify(user_totp):
        print("TOTP is valid!")
    else:
        print("Invalid TOTP.")

if __name__ == "__main__":
    # secret_key = input("Enter your secret key: ")
    secret_key = "M6MX4TV2S4PMQYASP2DYUPS2UAG7O4UW"
    verify_totp(secret_key)
```



University
of Glasgow

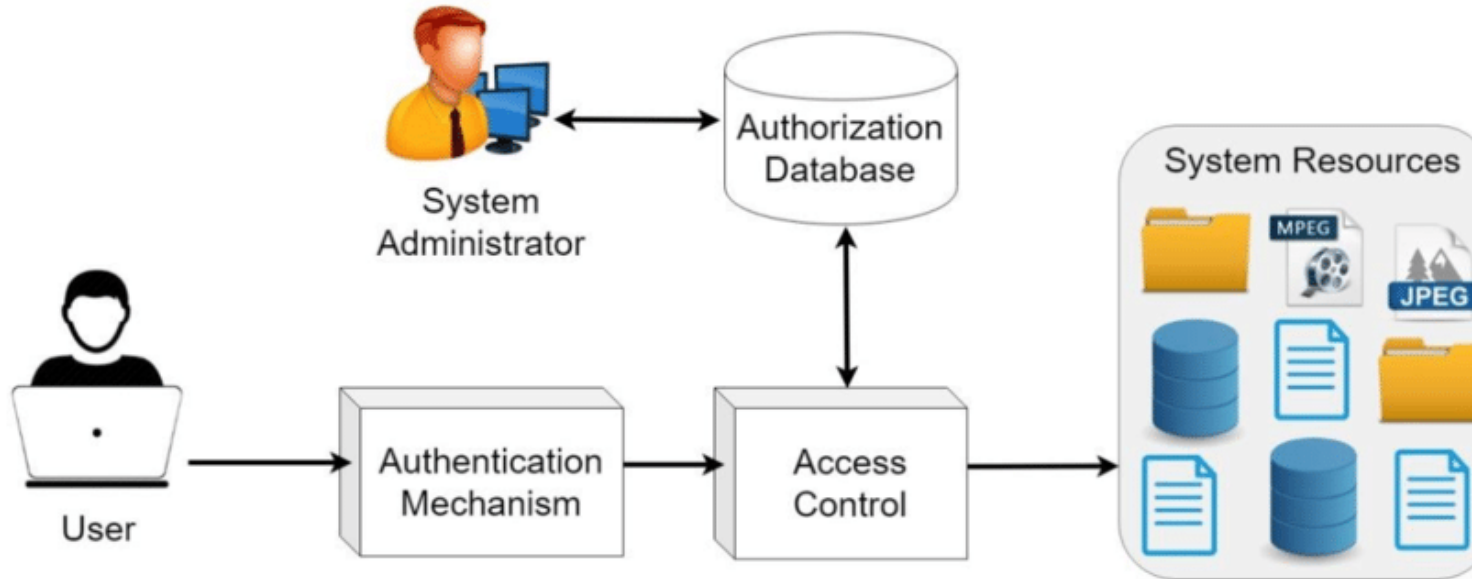
Authorization



Access Control'5

What you can do in the system?

Authorization



Authentication

Who you are/Whether you are allowed to used the system?



Authorization

- **Def**
 - Authorization is the process in a computer security system that specifies **access rights and privileges to resources**.
- It's a key component in ensuring that users or systems have appropriate access to resources based on their identity and role.



Type of Authorization

- Authorization Models (a.k.a, **Access Control Models**)
- **Role-based Access Control (RBAC)**
 - Access rights are assigned based on the **user's role** within the organization.
- **Attribute-Based Access Control (ABAC)**
 - Decisions are based on **attributes** of the user, resource, and environment.
- **Discretionary Access Control (DAC)**
 - **Resource owners** decide on access rights.
- **Mandatory Access Control (MAC):**
 - Access decisions are based on **predetermined security labels** assigned to users and data.



Discretionary Access Control (DAC)



Discretionary Access Control Model (DAC)

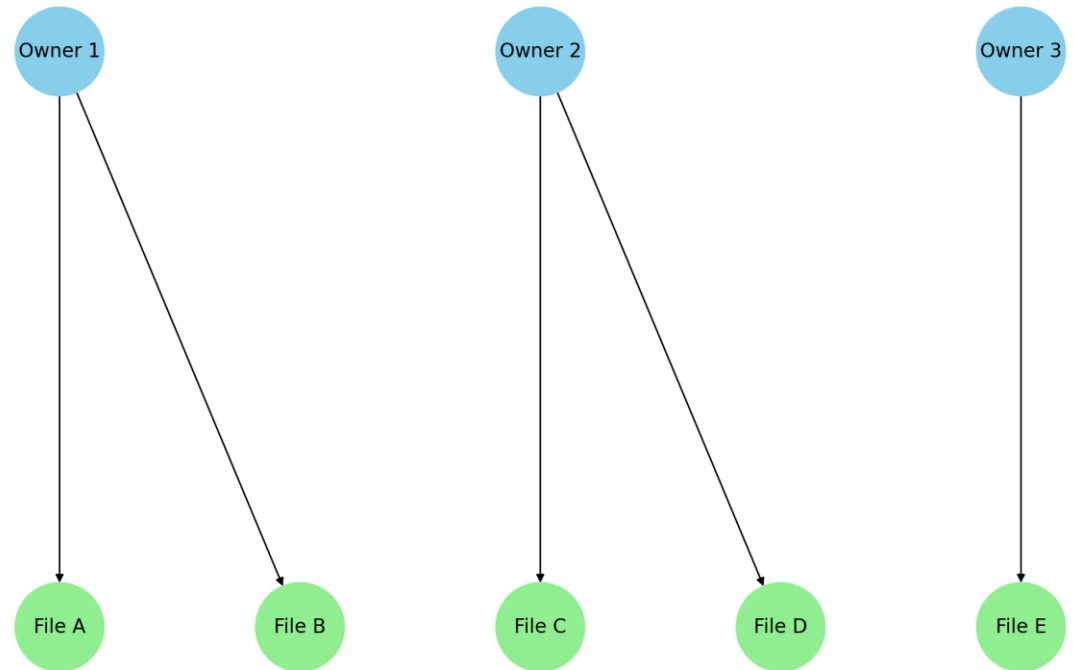
- **Discretionary**
- Discretion
 - Individual choice or judgment
 - (<https://www.merriam-webster.com/dictionary/discretion>)



Discretionary Access Control (DAC)

- **Definition:** DAC is a type of access control that allows owners to grant or restrict access to resources based on **the identity of subjects and/or groups**.
- **Key Feature:** In DAC systems, the control of access is left to the discretion of the individual users (or subjects) rather than being determined by a strict policy defined by the organization.

Discretionary Access Control (DAC) - Owner Controlled Access





How DAC Works?

- Implementation Method
 - DAC employs Access Control Lists (**ACLs**) for defining permissions.
- Assignment of Permissions
 - Resource owners allocate access rights using ACLs, which specify permissions for individual users and groups.
- Advantages of Using DAC
 - Offers a high degree of flexibility.
 - Simplifies the management of permission configurations.

How DAC Works?'2

- Administration Process
 - Security administrators create distinct profiles for resources or collections of resources.
 - They are responsible for the ongoing maintenance and revision of ACLs associated with these profiles.
- Real-World Applications
 - Commonly utilized for managing file permissions in various operating systems, including but not limited to Windows and UNIX.

▼ Name & Extension:

SSE-Lecture3.pptx

☐ Hide extension

▼ Comments:

▼ Open with:

 Microsoft PowerPoint (default)

Use this application to open all documents like this one.

Change All...

▼ Preview:



▼ Sharing & Permissions:

You have unknown access

Name	Privilege
 yutian.tang (M...	↕ Read & Write
 staff	↕ Read only
 everyone	↕ Read only

+

−

 ▼





Access Control List (ACL)

- Access control lists are **permission-based systems** that assign people in an organization different levels of access to resources (files and information).
- Common ACLs:
 - File System ACLs
 - Manage access to files and directories on a system, defining user permissions and privileges.
 - Networking ACLs
 - Control network access, dictating the types of traffic that can interact with the network and user permissions within it.



Advantages and Disadvantages of DAC

- **Advantages:**
 - **Flexibility:** Owners tailor permissions to user needs.
 - **User Control:** End-users manage their own access rights.
 - **Ease of Use:** Simple to implement and manage, ideal for small organizations. (small to medium-size organizations)
 - **Cost-Effective:** Lower costs due to simplicity



Advantages and Disadvantages of DAC

- **Disadvantages:**

- **Security Vulnerabilities:** Susceptible to "Trojan horse" and other attacks due to broad user control. → *if the admin (central authority) is compromised by a Trojan horse,...*
- **Complex Management:** Difficult to manage effectively as the number of users or access rights increase.
- **Administrative Challenges:** Keeping privileges updated for many users can lead to errors



University
of Glasgow

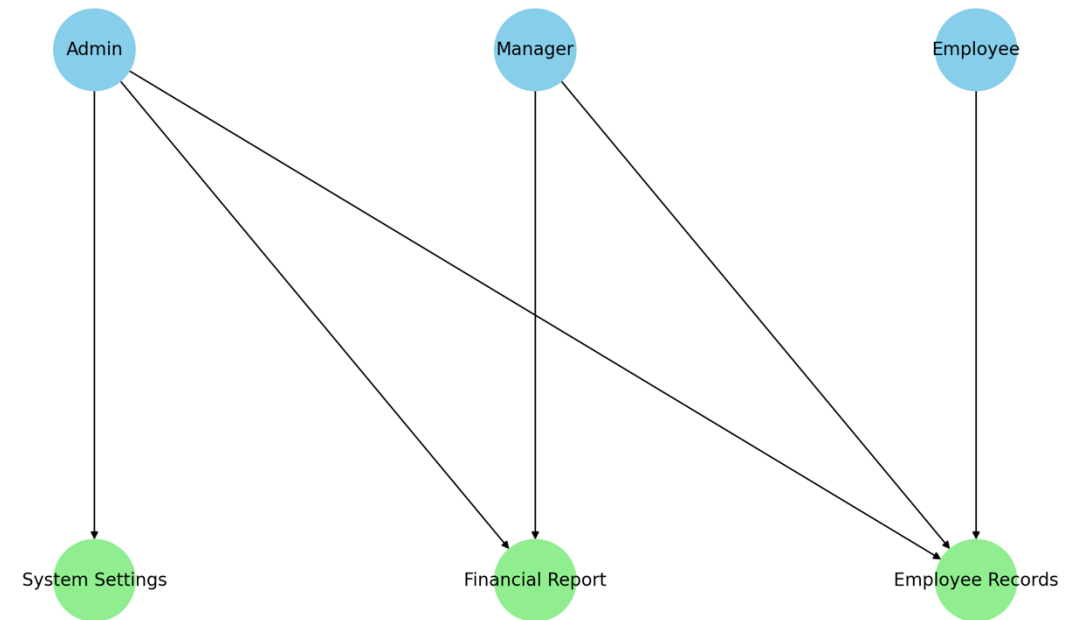
Role-based Access Control (RBAC)



Role-Based Access Control (RBAC)

- **Definition:** RBAC is an approach to restricting system access to authorized users based on their role within an organization.
- **Roles** are assigned based on job competencies, authority, and responsibility levels.
- **Users** are granted permissions for roles, not individually, simplifying the permissions assignment.

Role-Based Access Control (RBAC) Illustration





Role-Based Access Control (RBAC)

- **Role Definitions:**
 - Examples of roles like Administrator, Manager, and Employee, each with specific access rights.
- **Role Assignments:**
 - Process of assigning roles to users based on their roles/job function.
- **Dynamic Access Control:**
 - How roles can be adjusted, added, or removed to reflect changes in an organization.



Role-Based Access Control (RBAC)

- **Advantages:**

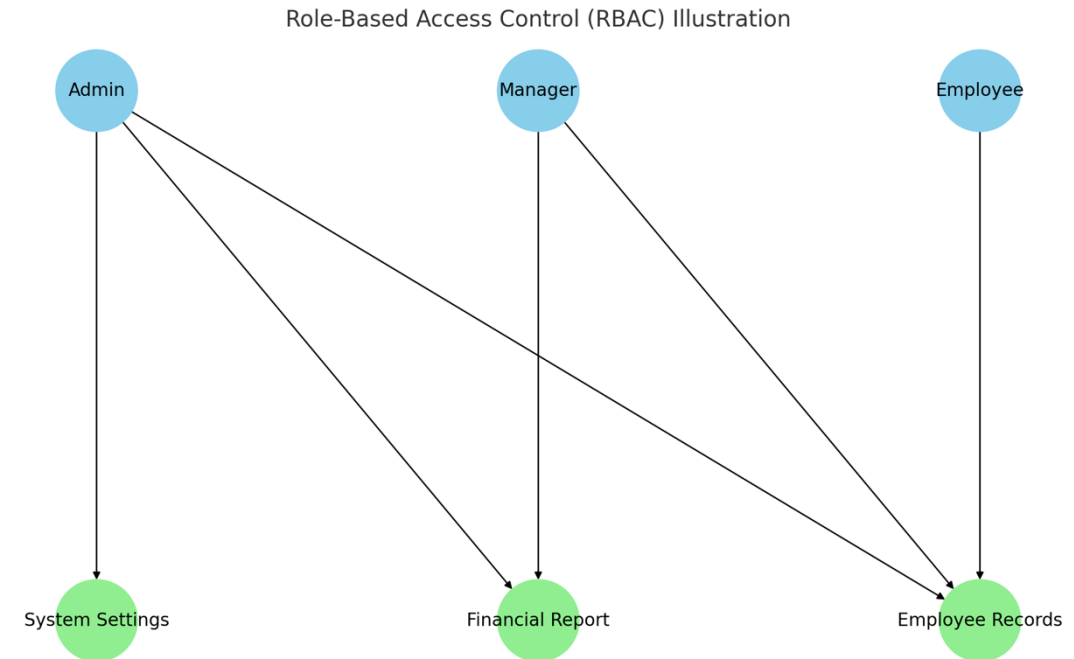
- Simplified Management: Easier to manage permissions as they are role-based.
- Enhanced Security: Limits unnecessary access, reducing potential for abuse.
- Scalability: Adapts easily to growing or changing organizations.

- **Considerations:**

- Role Definition: The importance of accurately defining roles.
- Role Creep: Avoiding accumulation of unnecessary privileges.

Role-Based Access Control (RBAC): Example

- The blue nodes represent different roles within an organization: Admin, Manager, and Employee.
- The green nodes are resources or systems that these roles can access: System Settings, Financial Report, and Employee Records.
- The arrows show the access permissions based on roles.
- For example,
 - the admin has access to all resources
 - the manager has access to the Financial Report and Employee Records, and
 - the employee can only access to the Employee Records.





Attribute-Based Access Control (ABAC)



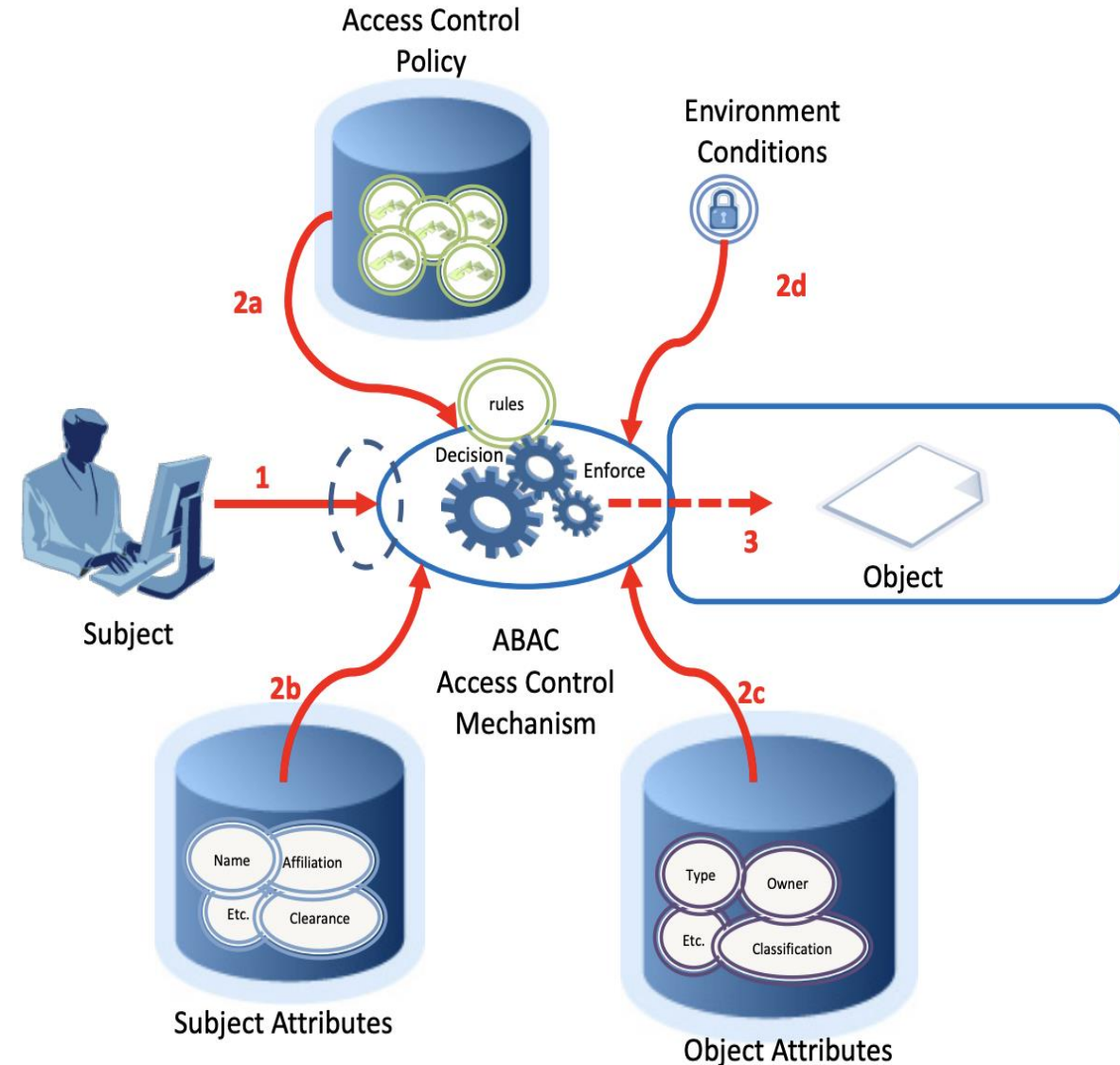
Attribute-Based Access Control (ABAC)

- **Definition:**
 - An access control method where subject requests to perform operations on objects are granted or denied based on assigned **attributes of the subject, assigned attributes of the object, environment conditions, and a set of policies** that are specified in terms of those attributes and conditions.
- Referred to as Policy-Based Access Control (PBAC) or Claims-Based Access Control (CBAC).
- **Key Components:** **Attributes** are properties associated with users, resources, or the operational context.
- **Functionality:** Access decisions are made by evaluating policies against the attributes.



Working of ABAC

1. Subject requests access to object
2. Access Control Mechanism evaluates
 - a) Rules,
 - b) Subject Attributes,
 - c) Object Attributes, and
 - d) Environment Conditions to compute a decision
3. Subject is given access to object if authorized





Working of ABAC

- Policy-Based Decision Making
- In ABAC, access decisions are enforced based on policies that evaluate the attributes of these components.
- These policies are essentially boolean functions that can dynamically change to account for organizational policy shifts.
- For example, a policy might state that employees in the communications department have read and edit access to media strategies for the marketing unit they represent.



Working of ABAC'2

- When a user requests access to a resource, the ABAC system analyzes the request's attribute values and matches them with established policies.
- If the attributes of the subject, action, resource, and environment align with the rules **defined in the policy**, the system grants access.



Attribute-Based Access Control (ABAC)

- **Advantages:**
 - Granular Access Control: Fine-grained security decisions based on multiple factors.
 - Scalability and Flexibility: Effective in dynamic environments like cloud services.
 - Context-Aware Security: Considers situational factors in access control.



Attribute-Based Access Control (ABAC)

- **Challenges:**

- Complexity in Implementation: Requires thorough planning and attribute management.
- Policy Development: Necessitates careful crafting of policies to avoid conflicts and gaps.



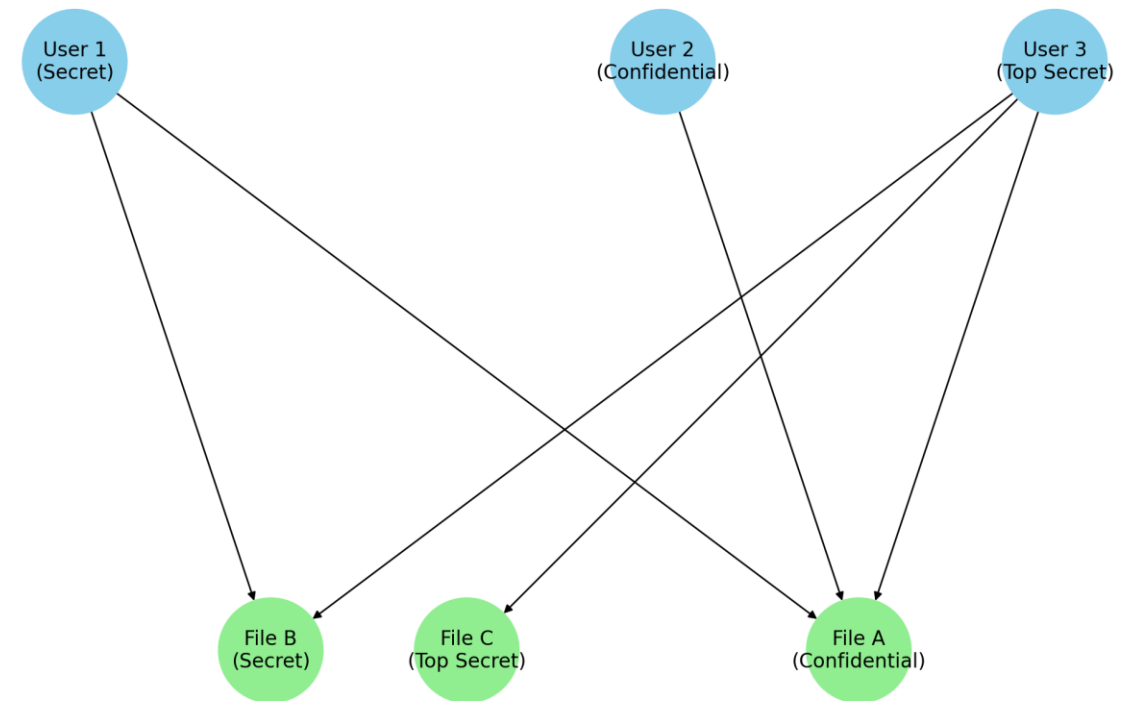
Mandatory Access Control (MAC)



Introduction to Mandatory Access Control (MAC)

- **Definition:** MAC is a type of access control where access rights are regulated by a central authority based on **predefined security labels**.
- **How it Works:** Users and data are assigned security labels (like **confidential, secret, top secret**).
- Access decisions are made based on these **labels and the clearance level of the users**.

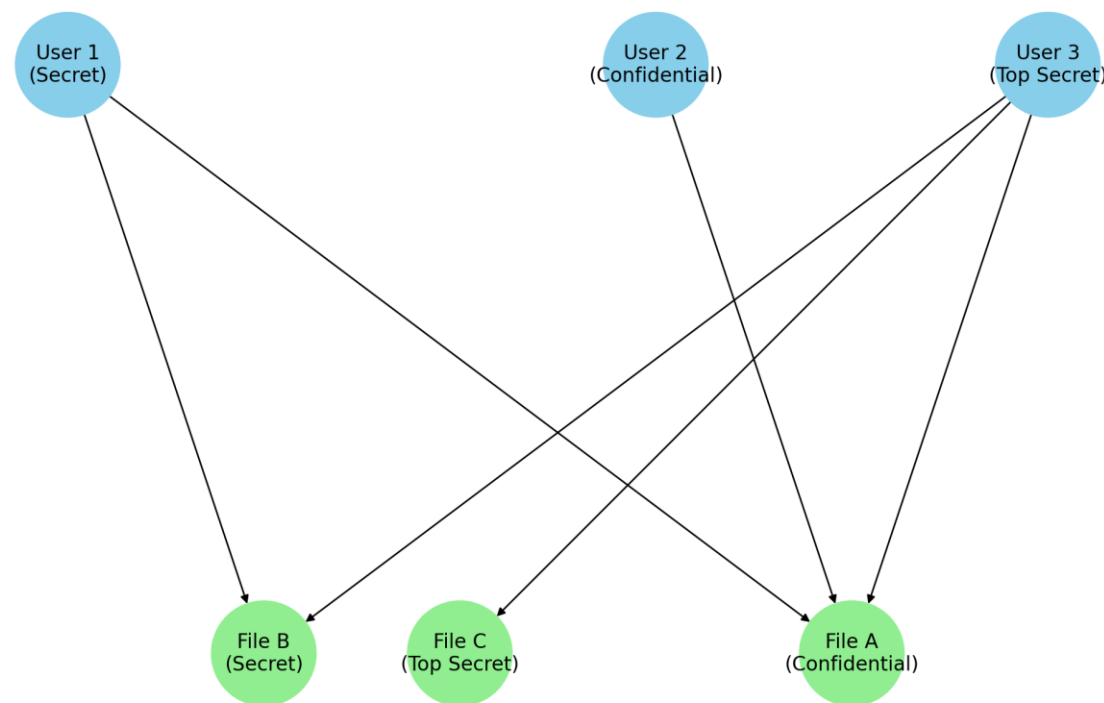
Mandatory Access Control (MAC) Illustration with Security Clearances



Introduction to Mandatory Access Control (MAC)

- Simply put, users can only access the data if their clearance level is appropriate for the data's security label.
- **For example,**
 - User 1 has a 'Secret' clearance level.
 - User 2 holds a 'Confidential' clearance level.
 - User 3 is granted a 'Top Secret' clearance level.
- **Resources:**
 - File A is classified as 'Confidential'.
 - File B carries a 'Secret' classification.
 - File C is marked as 'Top Secret'.

Mandatory Access Control (MAC) Illustration with Security Clearances

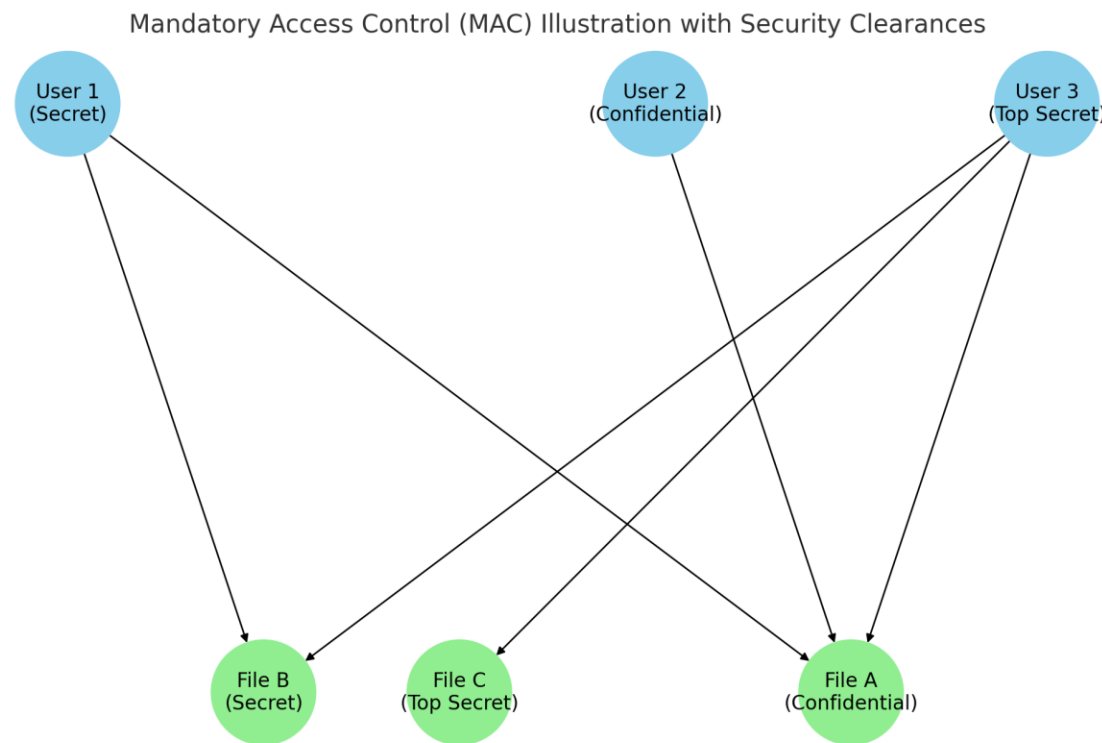


Introduction to Mandatory Access Control (MAC)

Confidential < Secret < Top Secret

For example,

- User 1 has a 'Secret' clearance level.
- **Resources:**
 - File A is classified as 'Confidential'.
 - File B carries a 'Secret' classification.
 - File C is marked as 'Top Secret'.
- User 1, with '**Secret**' clearance, can access File B, and File A, but not File C.
- User 2, with '**Confidential**' clearance, can only access File A.
- User 3, with '**Top-Secret**' clearance, can access all files.





Advantages and Disadvantages of MAC

- **Advantages:**
 - **High Security:**
 - Very effective in protecting sensitive information.
 - **Standardized Access Control:**
 - Provides a uniform access control mechanism that is not subject to user discretion.
 - **Reduced Risk of Unauthorized Access:**
 - Due to strict policy enforcement.



Advantages and Disadvantages of MAC

- **Disadvantages:**

- **Lack of Flexibility:** Not suitable for dynamic environments where access needs frequently change.
- **Complex Administration:** Requires meticulous setup and maintenance of security labels and clearance levels.
- **Potential for Over-Classification:** Can lead to inefficiencies, as too much data might be restricted.

References & Acknowledgement

- Jurcut, Anca D., Pasika Ranaweera, and Lina Xu. "Introduction to IoT security." IoT security: advances in authentication (2020): 27-64.
- <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/security/a-guide-to-data-security/security/passwords-in-online-services/>
- <https://owasp.org/www-project-proactive-controls/v3/en/c7-enforce-access-controls.html>
- <https://owasp.org/www-project-proactive-controls/v3/en/c6-digital-identity.html>
- <https://brightdata.com/blog/brightdata-in-practice/how-to-bypass-captcha-using-web-unlocker>
- <https://www.imperva.com/learn/application-security/what-is-captcha/>
- https://profsandhu.com/cs5323_s18/L9.pdf
- <https://csrc.nist.gov/CSRC/media/Presentations/Role-based-Access-Control-an-Overview/images-media/alvarez.pdf>



University
of Glasgow

Thank you!
Any Questions?